

The Krasnosel'skiĭ-Mann Iterative Method

Chapter 8: Two Applications

Based on Dong, Cho, He, Pardalos & Rassias

Chapter Outline

- 1 Recap and Coordinate Decomposition
- 2 8.1 The Asynchronous Parallel Coordinate Updates Method
- 3 8.2 The Cyclic Coordinate Update Algorithm
- 4 Conclusion: Where This Leaves Us

Notation You'll Need / Recap I

This chapter is about *scaling up* the Krasnosel'skii–Mann (KM) iteration you already know to problems where the unknown x lives in a huge-dimensional (or even infinite-dimensional) space $\mathcal{H}$, so large that updating *all* of x at every step is too expensive. The trick: split x into blocks and update one block ("coordinate") at a time.

Notation You'll Need / Recap II

Recap: The KM Iteration and Averaged Operators (Ch. 3–4)

For a nonexpansive operator $T : \mathcal{H} \rightarrow \mathcal{H}$ (i.e. $\|T(x) - T(y)\| \leq \|x - y\|$ for all x, y) with $\text{Fix}(T) \neq \emptyset$ (the fixed-point set is nonempty), write $S := \text{Id} - T$. Then

$$\text{Find } x^* \in \mathcal{H} \text{ such that } T(x^*) = x^* \quad \Longleftrightarrow \quad \text{Find } x^* \in \mathcal{H} \text{ such that } S(x^*) = 0.$$

With a constant relaxation parameter $\lambda_n \equiv \lambda \in (0, 1)$, the KM iteration $x_{n+1} = (1 - \lambda)x_n + \lambda T(x_n)$ can be rewritten equivalently as

$$x_{n+1} = x_n - \lambda S(x_n) \quad \text{for each } n \geq 0.$$

This is the form we build on in this chapter: at each step we move x_n by $-\lambda$ times the “residual” $S(x_n)$.

Notation

- $\mathcal{H}$ (calligraphic H) is a Hilbert space; Id is the identity operator, $\text{Id}(x) = x$.
- $S = \text{Id} - T$ measures how far T moves a point; $S(x^*) = 0$ exactly when x^* is a fixed point of T .
- λ (lambda) is the relaxation parameter from Chapter 7 — it controls step size and can be tuned for faster convergence.

The Big Idea of Chapter 8: Coordinate Splitting

Instead of moving the whole point $x \in \mathcal{H}$ at once, split it into m blocks $x = (x_1, \dots, x_m)$ and update just *one block* x_i at a time — cycling through all of them in a fixed order (**cyclic**), or letting different processors update different blocks in parallel/at random without waiting for each other (**asynchronous parallel**). This is exactly what you do with coordinate descent for a huge linear system: instead of solving for every variable simultaneously, you update x_1 , then x_2 , then $x_3, \dots$, one at a time, and it still converges to the solution.

Block/Coordinate Decomposition

Let $\mathcal{H}_1, \dots, \mathcal{H}_m$ be Hilbert spaces and $\mathcal{H} := \mathcal{H}_1 \times \dots \times \mathcal{H}_m$ their Cartesian product. Write $x = (x_1, \dots, x_m) \in \mathcal{H}$ with $x_i \in \mathcal{H}_i$ the i -th *block* (or *coordinate*) of x . For $S = \text{Id} - T : \mathcal{H} \rightarrow \mathcal{H}$, the i -th *coordinate operator* is

$$S_i(x) := (0, \dots, 0, (S(x))_i, 0, \dots, 0) \quad \text{for all } x \in \mathcal{H},$$

i.e. $S_i(x)$ keeps only the i -th block of $S(x)$ and zeroes out every other block. Updating x by $-\lambda S_i(x)$ therefore changes *only* x_i , leaving every other block untouched.

Cyclic Order vs. Asynchronous Parallel

- **Cyclic:** a single process visits coordinates $1, 2, \dots, m$ in a fixed order (or a permutation chosen once), applying one coordinate update per step; one full pass through all m coordinates is called an *epoch*.
- **Asynchronous parallel:** several agents (processors) update different coordinates *concurrently*, *without waiting for one another*. Because agent updates interleave unpredictably, an agent may read a *stale* (slightly out-of-date, or delayed) copy of the coordinates it does not own — it does not see the very latest values written by other agents. The maximum number of such missed updates is a *delay* τ .

8.1 Why Asynchronous Parallel Updates? I

Synchronous parallel algorithms make all processors update simultaneously and then force everyone to wait at a barrier before the next round starts — the slowest processor holds everyone else back. The *asynchronous parallel* approach removes the barrier: every agent runs continuously with little idling, and iterations become *disordered* (an agent may update a coordinate without having the newest information about the other coordinates).

8.1 Why Asynchronous Parallel Updates? II

Why This Matters

- Async-parallel iteration was first introduced by Chazan and Miranker (1969) and has since been applied to linear systems, nonlinear problems, differential equations, consensus problems, and optimization.
- Async-parallel systems are easy to extend with new agents on the fly, and are more tolerant of computing faults and communication glitches than synchronous systems (no single point that everyone must wait on).
- Peng et al. introduced **ARock**, an algorithmic framework for asynchronous parallel coordinate updates for the fixed-point problem of nonexpansive mappings — ARock is itself an important application of the KM iteration.

8.1 Why Asynchronous Parallel Updates? III

Setting

Let $\mathcal{H}_1, \dots, \mathcal{H}_m$ be Hilbert spaces, $\mathcal{H} := \mathcal{H}_1 \times \dots \times \mathcal{H}_m$. For a nonexpansive $T : \mathcal{H} \rightarrow \mathcal{H}$, consider the fixed-point problem

$$\text{Find } x^* \in \mathcal{H} \text{ such that } T(x^*) = x^*. \quad (8.1)$$

As before, $S = \text{Id} - T$ turns this into finding a zero of S : $0 = S(x^*)$.

8.1 Why Asynchronous Parallel Updates? IV

The ARock Coordinate Update

Let $(p_1, \dots, p_m)$ be a probability distribution ($p_i > 0$, $\sum_i p_i = 1$). At step n , a coordinate $i_n \in \{1, \dots, m\}$ is randomly selected with $\text{Prob}(i_n = i) = p_i$, and only $x_{i_n} \in \mathcal{H}_{i_n}$ is updated:

$$x_{n+1} = x_n - \frac{\eta_n}{m p_{i_n}} S_{i_n}(\hat{x}_n) \quad \text{for each } n \geq 0, \quad (8.3)$$

where $\eta_n > 0$ is a scalar step size and $m p_{i_n}$ normalizes for nonuniform selection probabilities. In the *uniform* case $p_i \equiv 1/m$, we have $m p_{i_n} \equiv 1$, and (8.3) simplifies to

$$x_{n+1} = x_n - \eta_n S_{i_n}(\hat{x}_n) \quad \text{for each } n \geq 0. \quad (8.4)$$

8.1 Why Asynchronous Parallel Updates? V

Notation

- x_n is the state of x in *global memory* just before update (8.3) is applied.
- $\hat{x}_n$ is what an agent actually *reads* from global memory into its local cache and applies S_{i_n} to. In a *synchronous*-parallel algorithm $\hat{x}_n = x_n$ exactly; in ARock, because other agents may write to x concurrently, $\hat{x}_n$ can differ from x_n — this staleness is the key difference between sync- and async-parallel algorithms.

8.1 Why Asynchronous Parallel Updates? VI

Quantifying the Staleness (Delay)

The relationship between $\hat{x}_n$ and x_n is

$$\hat{x}_n = x_n + \sum_{d \in J(n)} (x^d - x^{d+1}) \quad \text{for each } n \geq 1, \quad (8.5)$$

where $J(n) \subseteq \{n-1, \dots, n-\tau\}$ and $\tau \in \mathbb{N}^+$ is the *maximum delay*: the maximum number of other updates to x that can occur during the computation of a single update (8.3). Larger τ means more staleness is tolerated, at the cost of a more delicate convergence analysis.

8.1 Algorithm 9 (ARock): Step by Step I

Intuition

Think of m workers, each responsible for one block of coordinates. Each worker, whenever it is free, grabs a random coordinate index, reads the (possibly slightly out-of-date) values of the other coordinates it needs, computes an update, and writes it back — all without synchronizing with anyone else.

8.1 Algorithm 9 (ARock): Step by Step II

Algorithm 9 – ARock: Framework for Asynchronous Parallel Coordinate Updates

Input: $x_0 \in \mathcal{H}$, a horizon $N > 0$, a distribution $(p_1, \dots, p_m) > 0$ with $\sum_{i=1}^m p_i = 1$; global iteration counter $n \leftarrow 0$.

While $n < N$, **every agent asynchronously does the following:**

- ① **Select a coordinate.** Choose $i_n \in \{1, \dots, m\}$ at random with $\text{Prob}(i_n = i) = p_i$.
- ② **Update that coordinate only.** Read the (possibly stale) local copy $\hat{x}_n$ and perform an update to x_{i_n} according to (8.3): $x_{n+1} = x_n - \frac{\eta_n}{m p_{i_n}} S_{i_n}(\hat{x}_n)$, holding every other block fixed.
- ③ **Advance the counter.** Update the global counter $n \leftarrow n + 1$, and continue — with no barrier, no waiting for other agents.

Key contrast with the cyclic version (Algorithm 10, Section 8.2): in the cyclic algorithm every coordinate is updated exactly once per epoch, in a known order, using perfectly up-to-date information. In ARock, coordinates are updated in *random, overlapping* order across agents, and the information used, $\hat{x}_n$, may lag behind the true global state x_n by up to τ updates — this staleness/delay is the price paid for removing synchronization barriers.

8.1 Algorithm 9 (ARock): Step by Step III

Two Standing Assumptions

- ① **Atomic coordinate update:** a coordinate is not further broken into smaller pieces during an update — it is written all at once (no partial/torn writes).
- ② **Bounded maximal delay τ :** during any update cycle of an agent, x in global memory is updated at most τ times by other agents.

For the analysis of Algorithm 9 we further need: (a) random coordinate selection, (b) a finite maximal delay τ (not essential — an infinite delay with a light tail is also allowed), and (c) bounded relaxation parameters (standard).

When Is This Worth It?

The coordinate update (8.3) is only computationally worthwhile if $S_i(x)$ is much cheaper to compute than the full $S(x)$ — e.g. for systems of linear equations, decentralized consensus optimization, and feasibility problems, where a single coordinate can be updated from only its neighbors' data. Otherwise, it is preferable to just apply the full KM update $x_{n+1} = x_n - \lambda S(x_n)$.

8.1 Convergence of Algorithm 9 I

Probabilistic Setup

Let $(\Omega, \mathcal{F}, P)$ be the underlying probability space and let $\mathcal{X}_n := \sigma(x_0, \hat{x}_1, x_1, \dots, \hat{x}_n, x_n)$ be the smallest σ -algebra generated by the history up to step n . Assume $p_{\min} := \min_i p_i > 0$ and that the coordinate selection is conditionally i.i.d.: $\text{Prob}(i_n = i \mid \mathcal{X}_n) = \text{Prob}(i_n = i) = p_i$.

Lemma 8.1 (One-Step Expected-Distance Bound)

Let $T : \mathcal{H} \rightarrow \mathcal{H}$ be nonexpansive with $\text{Fix}(T) \neq \emptyset$ and $\{x_n\}$ generated by Algorithm 9. Then for any $x^* \in \text{Fix}(T)$,

$$\mathbb{E}(\|x_{n+1} - x^*\|^2 \mid \mathcal{X}_n) \leq \|x_n - x^*\|^2 + \frac{\gamma}{m} \sum_{d \in J(n)} \|x_d - x_{d+1}\|^2 + \frac{1}{m} \left(\frac{|J(n)|}{\gamma} + \frac{1}{mp_{\min}} - \frac{1}{\eta_n} \right) \|x_n - \bar{x}_{n+1}\|^2,$$

where $\gamma > 0$ is an arbitrary constant and $\bar{x}_{n+1} = x_n - \eta_n S(\hat{x}_n)$.

8.1 Convergence of Algorithm 9 II

Theorem 8.1 (Weak/Strong Convergence)

Let T be nonexpansive with $\text{Fix}(T) \neq \emptyset$. If $\eta_n \in [\eta_{\min}, \frac{c m p_{\min}}{2\tau\sqrt{p_{\min}+1}}]$ for some $\eta_{\min} > 0$ and $0 < c < 1$, then $\{x_n\}$ from Algorithm 9 converges weakly to a fixed point of T with probability one. This becomes strong convergence if $\mathcal{H}$ is finite-dimensional, and *strong* convergence to a fixed point holds (a.s.) if T is demicompact. Notably, this weak-convergence result requires only that T be nonexpansive with a fixed point — no monotonicity is needed.

8.1 Convergence of Algorithm 9 III

Reading the Step-Size Bound

In the uniform case $p_{\min} \equiv p_i \equiv 1/m$, the bound $\frac{cm p_{\min}}{2\tau\sqrt{p_{\min}}+1}$ simplifies to $\frac{1}{1+2\tau/\sqrt{m}}$. If the maximal delay is no larger than the square root of the number of coordinates, i.e. $\tau = O(\sqrt{m})$, the bound is $O(1)$ — a healthy, non-shrinking step size is allowed even with many agents.

8.1 Convergence of Algorithm 9 IV

Theorem 8.2 (Linear Convergence under Quasi-Strong Monotonicity)

Assume S is quasi- μ -strongly monotone with $\mu > 0$ (this holds, e.g., for the KM operators arising from minimizing a closed proper convex function plus a strongly convex, Lipschitz-differentiable function). Let $\beta \in (0, 1)$ and let $\{x_n\}$ be generated by Algorithm 9 with a *constant* relaxation parameter $\eta \in (0, \min\{\underline{\eta}_1, \underline{\eta}_2\}]$, where

$$\underline{\eta}_1 = \left(1 - \frac{1}{\rho}\right) \frac{m\sqrt{p_{\min}}}{8} \cdot \frac{\rho^{1/2} - 1}{\rho^{(\tau+1)/2} - 1} \quad \text{for some } \rho > 1, \quad \underline{\eta}_2 = \frac{-b + \sqrt{b^2 + 4(1 - \beta)a}}{2a},$$
$$a = \frac{2\beta\mu\tau}{m^2 p_{\min}} \cdot \frac{\rho(\rho^\tau - 1)}{\rho - 1}, \quad b = \frac{1}{mp_{\min}} + \frac{2}{m} \sqrt{\frac{\rho(\rho^\tau - 1)\tau}{(\rho - 1)p_{\min}}}.$$

Then $\mathbb{E}(\|x_n - x^*\|^2) \leq \left(1 - \frac{\beta\mu\eta}{m}\right)^n \|x_0 - x^*\|^2$, where x^* is the unique fixed point of T .

8.1 Convergence of Algorithm 9 V

A Clean Asymptotic Regime

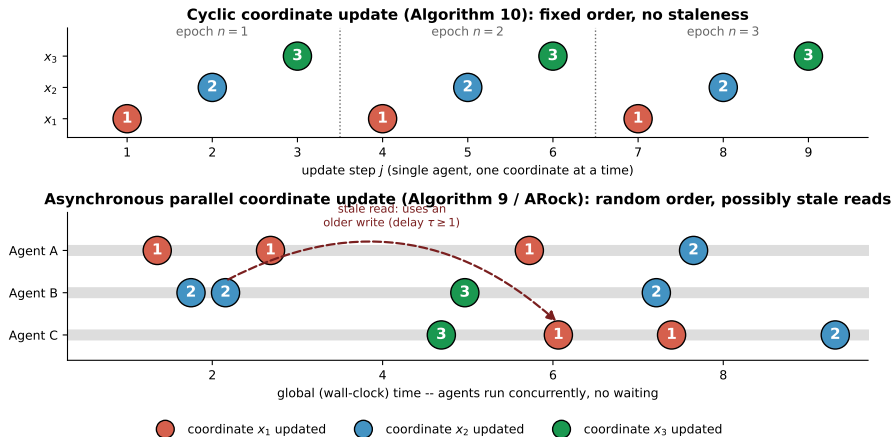
Suppose i_n is uniform, $p_{\min} \equiv 1/m$, and both m, τ are large with $\sqrt{\rho} = 1 + \frac{1}{\tau}$. One can check $\underline{\eta}_1 = O(\sqrt{m}/\tau^2)$ and $\underline{\eta}_2 = O(\sqrt{m}/\tau)$. Hence if $\tau = O(m^{1/4})$, the relaxation parameter in Theorem 8.2 can be taken as $\eta = \overline{O}(1)$ — so the linear convergence rate survives even with substantial delay, as long as the delay does not grow faster than $m^{1/4}$.

Beyond ARock: Inertial Extensions

Combining asynchronous algorithms with inertial acceleration (recall Chapters 5–6), Stathopoulos and Jones proposed an inertial asynchronous and parallel KM iteration; Heaton and Censor introduced an asynchronous sequential inertial framework for finding a point in the intersection of fixed-point sets of a finite collection of operators.

8.1 Illustration: Cyclic vs. Asynchronous Schedules

On a toy example with $m = 3$ coordinates, the diagram below contrasts the two update patterns: a fixed repeating order (top) vs. three concurrent agents with a stale read (bottom, dashed arrow, delay $\tau \geq 1$).



8.1 Python (1/2): ARock-Style Async Updates – Setup

```
1 import numpy as np
2
3 # Running example:  $Ax = b$  (SPD,  $m = 3$  coordinates)
4 A = np.array([[4., 1., 1.], [1., 4., 1.], [1., 1., 4.]])
5 b = np.array([6., 9., 12.])
6 x_star = np.linalg.solve(A, b)      # true fixed point (0.5, 1.5, 2.5)
7
8 def S(x):
9     return A @ x - b                #  $S = Id - T$  (residual operator)
10
11 def arock_style(x0, n_updates, eta, tau=2, p=None, seed=0):
12     """Single-machine simulation of ARock (Algorithm 9): random
13     coordinate choice + a simulated stale read of size  $\leq \tau$ ."""
14     rng = np.random.default_rng(seed)
15     m = len(x0)
16     p = p if p is not None else np.ones(m) / m
17     history = [x0.copy()]           # global memory  $x_0, x_1, x_2, \dots$ 
18     x = x0.copy()
```

Listing 1: Running example and ARock scaffold (Algorithm 9)

8.1 Python (2/2): ARock-Style Async Updates – Loop

```
1  for n in range(n_updates):
2      i_n = rng.choice(m, p=p)           # Step 1: random coordinate
3      delay = rng.integers(0, tau + 1)   # simulated staleness
4      stale_idx = max(0, len(history) - 1 - delay)
5      x_hat = history[stale_idx].copy()   # Step 2: possibly stale read
6      S_i = np.zeros(m)
7      S_i[i_n] = S(x_hat)[i_n]
8      x = x - (eta / (m * p[i_n])) * S_i  # ARock update (8.3)
9      history.append(x.copy())           # Step 3: advance counter
10     return np.array(history)
11
12 hist = arock_style(np.zeros(3), n_updates=300, eta=0.2, tau=2)
13 print("final iterate:", hist[-1], " true x*:", x_star)
14 print("error:", np.linalg.norm(hist[-1] - x_star))
```

Listing 2: ARock update loop (Steps 1–3) and usage

8.2 Setup: Cyclic Coordinate Updates I

Intuition

The cyclic coordinate update is the “single-processor” analogue of ARock: instead of a random, interleaved, possibly-stale schedule across many agents, one process visits every coordinate exactly once, in a fixed (or once-shuffled) order, always using the freshest information available. This is precisely coordinate descent / Gauss–Seidel-type sweeps, now cast in the KM framework.

Setup

Let $T : \mathcal{H} \rightarrow \mathcal{H}$ be nonexpansive and $S = \text{Id} - T$. The fixed-point problem (8.1) is equivalent to

$$\text{Find } x^* \in \mathcal{H} \text{ such that } S(x^*) = 0. \quad (8.6)$$

Assume x has m blocks $x_i \in \mathcal{H}_i$ for $i = 1, \dots, m$, with Cartesian product $\mathcal{H} := \mathcal{H}_1 \times \dots \times \mathcal{H}_m$. As in 8.1, the i -th coordinate operator is

$$S_i(x) = (0, \dots, (S(x))_i, \dots, 0) \quad \text{for all } x \in \mathcal{H}.$$

8.2 Setup: Cyclic Coordinate Updates II

Lipschitz Constants of the Coordinate Operators

Since T is nonexpansive, S is 2-Lipschitz (recall Lemma 7.3): $\|S(x) - S(y)\| \leq 2\|x - y\|$ for all $x, y \in \mathcal{H}$. Since $\|S_i(x) - S_i(y)\| \leq \|S(x) - S(y)\|$, each S_i is 2-Lipschitz too — but the Lipschitz constant of a *single* coordinate operator S_i can be much smaller than 2. Denote by L_i the Lipschitz constant of S_i :

$$\|S_i(x) - S_i(y)\| \leq L_i \|x - y\| \quad \text{for all } x, y \in \mathcal{H}, \quad L := \max_{1 \leq i \leq m} L_i \leq 2.$$

L will control the admissible step sizes in Theorem 8.5 below.

8.2 Algorithm 10: The Cyclic Coordinate Update, Step by Step I

Algorithm 10 (Chow et al.) – The Cyclic Coordinate Update

input: $x_0 \in \mathcal{H}$

for $n = 1, 2, \dots$, **do**

- ① **Pick an order.** Set $(i_1, i_2, \dots, i_m)$ as a permutation of $(1, 2, \dots, m)$ (either no permutation, random shuffling, or a greedy ordering); choose a relaxation parameter $\lambda_n > 0$; initialize $y_0 \leftarrow x_{n-1}$.
- ② **Sweep through all m coordinates.** For $j = 1, 2, \dots, m$: apply the averaged-operator update to coordinate i_j only, holding every other coordinate fixed,

$$y_j \leftarrow y_{j-1} - \lambda_n S_{i_j}(y_{j-1}).$$

- ③ **Close the epoch.** After $j = m$, set $x_n \leftarrow y_m$, and move to the next epoch $n + 1$ (repeat Step 1 with a possibly new permutation).

8.2 Algorithm 10: The Cyclic Coordinate Update, Step by Step II

Coordinatewise view of Step 2: each inner iteration only touches the i_j -th block,

$$(y_j)_i = \begin{cases} (y_{j-1})_i - \lambda_n (S(y_{j-1}))_i, & \text{if } i = i_j, \\ (y_{j-1})_i, & \text{otherwise,} \end{cases} \quad \text{for each } i = 1, \dots, m.$$

8.2 Algorithm 10: The Cyclic Coordinate Update, Step by Step III

Notation

- n indexes the *outer loop* (an *epoch* = one full pass through all m coordinates); j indexes the *inner loop* (a single coordinate update within the epoch).
- $y_0, \dots, y_m$ are the intermediate states within epoch n ; y_0 starts from the previous epoch's result x_{n-1} , and y_m becomes x_n .
- λ_n is a single relaxation parameter shared by *all* m coordinate updates within epoch n (it may change from epoch to epoch).

Choosing the Order

The order of the m coordinates is fixed at the start of each epoch, and each epoch may choose independently. Typical choices: the *natural order* $(1, 2, \dots, m)$, a *random shuffle*, or a *greedy ordering* (placing a coordinate earlier if its Lipschitz constant L_i is larger). One may also shuffle only once, at the first epoch, and reuse that order for all later epochs. In general, shuffling avoids the worst-case ordering and may accelerate convergence.

8.2 Algorithm 10: The Cyclic Coordinate Update, Step by Step IV

On the Relaxation Parameter

Algorithm 10 uses the *same* λ_n for every coordinate within an epoch. But recall from Lemma 7.2 (Chapter 7) that the optimal relaxation parameter depends on the properties of the operator being applied — so it can be beneficial to replace λ_n by a coordinate- and epoch-dependent $\lambda_{n_{i,j}}$ instead of one shared value.

8.2 Convergence of Algorithm 10 I

Theorem 8.3 (Weak Convergence, Diminishing Step)

Let $T : \mathcal{H} \rightarrow \mathcal{H}$ be nonexpansive with $\text{Fix}(T) \neq \emptyset$. If the relaxation parameter is

$$\lambda_n = \frac{1}{n^{1/2}}, \tag{8.7}$$

then the sequence $\{x_n\}$ generated by Algorithm 10 converges weakly to a fixed point of T .

8.2 Convergence of Algorithm 10 II

A Relaxed Condition (Peng and Xu)

Condition (8.7) was relaxed to the pair of conditions

$$(G1) \sum_{n=0}^{+\infty} \lambda_n = +\infty \quad \text{and} \quad (G2) \sum_{n=0}^{+\infty} \lambda_n^3 < +\infty. \quad (8.8)$$

Condition (8.8) includes (8.7) as a special case. Note that any λ_n satisfying (8.7) or (8.8) is decreasing and $\rightarrow 0$ as $n \rightarrow \infty$ — this generally means *slow* convergence for Algorithm 10 in this regime.

Reading (G1)–(G2)

(G1) says the total step budget is unbounded — the iterates can travel arbitrarily far, so they are not prevented from reaching a distant fixed point. (G2) says the steps shrink fast enough (in cube) that the sequence still settles down rather than oscillating forever. This is the same flavor of trade-off as the Robbins–Monro step-size conditions for stochastic approximation.

8.2 Convergence of Algorithm 10 III

Theorem 8.4 (Residual Reduction Rate)

Under the setting of Theorem 8.3, the reduction rate of the *running minimal residual* is

$$\min_{j \leq n} \{ \|x_j - T(x_j)\|^2 \} = o\left(\frac{1}{\sqrt{n}}\right).$$

It follows that $\min_{j \leq n} \{ \|x_j - T(x_j)\| \} = o(1/n^{1/4})$.

8.2 Convergence of Algorithm 10 IV

Theorem 8.5 (Linear Convergence, Constant Step)

Assume $\text{Id} - T$ is quasi- μ -strongly monotone. Let $\{x_n\}$ be generated by Algorithm 10 with a constant relaxation parameter $\lambda_n \equiv \lambda$ satisfying

$$\lambda = \min \left\{ \frac{1}{4mL}, \frac{\mu}{4\sqrt{2}mL}, \frac{2mL}{17mL + 2\mu^2} \right\}. \quad (8.9)$$

Then $\|x_n - x^*\|^2 \leq \rho^n \|x_0 - x^*\|^2$, where $\rho = 1 - \frac{\lambda\mu^2}{2} < 1$ and x^* is the unique fixed point of T . When m and L/μ are large, one has $\lambda = O(\frac{\mu}{mL})$.

8.2 Convergence of Algorithm 10 V

Diminishing vs. Constant Step: What to Take Away

Theorem 8.3 needs no monotonicity but pays for it with a vanishing step size and only weak, sublinear-type guarantees (Theorem 8.4). Theorem 8.5 needs the extra structure of quasi-strong monotonicity of $\text{Id} - T$ but rewards it with a genuinely *linear* (geometric) convergence rate at a *fixed* step size — exactly the pattern you saw for relaxation parameters in Chapter 7, now applied coordinate-by-coordinate.

8.2 Running Example: A 3×3 Linear System via Cyclic Updates I

Setting Up the Example

Take $\mathcal{H} = \mathbb{R}^3$, $m = 3$ coordinates (one per scalar variable), and consider solving $Ax = b$ for a symmetric positive-definite A and vector b :

$$A = \begin{pmatrix} 4 & 1 & 1 \\ 1 & 4 & 1 \\ 1 & 1 & 4 \end{pmatrix}, \quad b = (6, 9, 12).$$

Let $f(x) = \frac{1}{2}x^\top Ax - b^\top x$ and $T(x) := x - \nabla f(x) = x - (Ax - b)$: since $A \succ 0$, T is (for a suitable step) an averaged/nonexpansive operator, and its fixed points are exactly the solutions of $Ax = b$. Here $S(x) = \text{Id}(x) - T(x) = Ax - b$ is the residual, and $S_i(x)$ keeps only the i -th residual component. Solving exactly, A and b above give the unique fixed point

$$x^* = (0.5, 1.5, 2.5).$$

8.2 Running Example: A 3×3 Linear System via Cyclic Updates II

Choosing λ_n : Exact Coordinate Minimization (Gauss–Seidel)

Every diagonal entry $A_{ii} = 4$, so taking $\lambda_n \equiv \lambda = 1/4 = 1/A_{ii}$ for every coordinate makes the update (Step 2 of Algorithm 10) *exactly zero out* the i -th residual: this is precisely a Gauss–Seidel sweep,

$$y_j \Big|_{i=i_j} = (y_{j-1})_{i_j} - \frac{1}{4} (Ay_{j-1} - b)_{i_j} = \frac{b_{i_j} - \sum_{k \neq i_j} A_{i_j k} (y_{j-1})_k}{A_{i_j i_j}}.$$

We use the natural order $(1, 2, 3)$ and start from $x_0 = (0, 0, 0)$.

8.2 Running Example: A 3×3 Linear System via Cyclic Updates III

Epoch 1 (by hand)

$$y_0 = x_0 = (0, 0, 0).$$

$$j=1 : S(y_0) = Ay_0 - b = (-6, -9, -12); \quad y_1 = y_0 - \frac{1}{4}(-6, 0, 0) = (1.5, 0, 0)$$

$$j=2 : Ay_1 - b = (0, -7.5, -10.5); \quad y_2 = y_1 - \frac{1}{4}(0, -7.5, 0) = (1.5, 1.875, 0)$$

$$j=3 : Ay_2 - b = (1.875, 0, -8.625); \quad y_3 = y_2 - \frac{1}{4}(0, 0, -8.625) = (1.5, 1.875, 2.15625)$$

So $x_1 = (1.5, 1.875, 2.15625)$, with $\|x_1 - x^*\| \approx 1.122$ (down from $\|x_0 - x^*\| = \|(0, 0, 0) - (0.5, 1.5, 2.5)\| \approx 2.958$).

8.2 Running Example: A 3×3 Linear System via Cyclic Updates IV

Epoch 2 (by hand)

$y_0 = x_1 = (1.5, 1.875, 2.15625)$.

$j=1 : Ay_0 - b = (4.03125, 2.15625, 0); \quad y_1 = (0.4921875, 1.875, 2.15625)$

$j=2 : Ay_1 - b = (0, 1.1484375, -1.0078125); \quad y_2 = (0.4921875, 1.587890625, 2.15625)$

$j=3 : Ay_2 - b = (-0.287109375, 0, -1.294921875); \quad y_3 = (0.4921875, 1.587890625, 2.47998046875)$

So $x_2 \approx (0.4922, 1.5879, 2.4800)$, and $\|x_2 - x^*\| \approx 0.0905$ — already within one-tenth of the fixed point after just two epochs, six coordinate updates total.

8.2 Running Example: A 3×3 Linear System via Cyclic Updates V

Continuing the Sweep: Epochs 3–5

Epoch n	x_1	x_2	x_3	$\ x_n - x^*\ $
0	0.0000	0.0000	0.0000	2.9580
1	1.5000	1.8750	2.1563	1.1220
2	0.4922	1.5879	2.4800	0.0905
3	0.4830	1.5092	2.5019	0.0194
4	0.4972	1.5002	2.5006	0.0029
5	0.4998	1.4999	2.5001	0.0003

The error shrinks geometrically epoch over epoch — consistent with Theorem 8.5's linear-rate guarantee (here the convergence is even faster than the generic worst case, because $\lambda = 1/A_{ii}$ performs *exact* coordinate minimization at each step, i.e., classical Gauss–Seidel on a diagonally dominant SPD matrix).

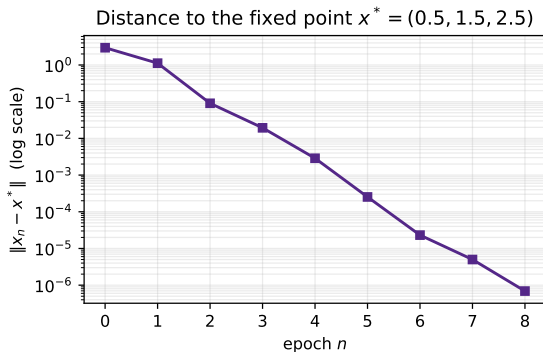
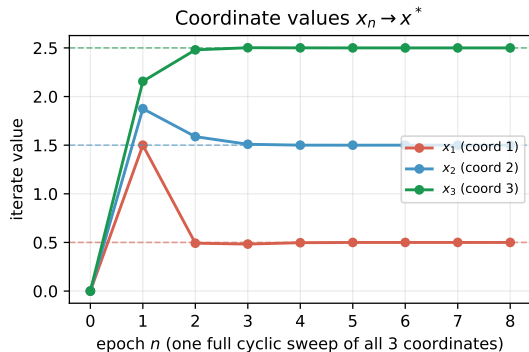
8.2 Running Example: A 3×3 Linear System via Cyclic Updates VI

A Brief Aside: Connection to the Book's Projection Example

The book's running example in earlier chapters used P_C , the projection onto the closed unit disk. A coordinate-update view of a projection onto a *box* $C = [a_1, b_1] \times \cdots \times [a_m, b_m]$ would similarly act coordinatewise: $(P_C(x))_i = \text{clip}(x_i, a_i, b_i)$, updated one i at a time. We use the linear-system example here instead, since it is the more natural vehicle for illustrating coordinate splitting with concrete numbers.

8.2 Illustration: Convergence of the Cyclic Update

Cyclic coordinate updates solving $Ax = b$ (running example: 3x3 SPD system, $b = (6, 9, 12)$)



Left: the three coordinates x_1, x_2, x_3 converge to $x^* = (0.5, 1.5, 2.5)$ (dashed lines) within a handful of epochs.

Right: the distance $\|x_n - x^*\|$ on a log scale, showing the geometric (linear) decay predicted by Theorem 8.5.

8.2 Python (1/2): Cyclic Coordinate Update – Setup

```
1 import numpy as np
2
3 A = np.array([[4., 1., 1.], [1., 4., 1.], [1., 1., 4.]])
4 b = np.array([6., 9., 12.])
5 x_star = np.linalg.solve(A, b)           # (0.5, 1.5, 2.5)
6
7 def S(x):
8     return A @ x - b                     #  $S = Id - T$ , the residual
9
10 def cyclic_coordinate_update(x0, n_epochs, lam, order=None):
11     """Algorithm 10: one full cyclic sweep through all  $m$  coordinates
12     is one epoch; order=None uses the natural order (1,...,m)."""
13     m = len(x0)
14     x = np.array(x0, dtype=float)
15     history = [x.copy()]
16     for n in range(1, n_epochs + 1):
17         seq = order if order is not None else range(m)
18         y = x.copy()
```

Listing 3: Running example and Algorithm 10 scaffold

8.2 Python (2/2): Cyclic Coordinate Update – Sweep

```
1         for i in seq:                                # Step 2: sweep coordinates
2             Sy = S(y)
3             Si = np.zeros(m)
4             Si[i] = Sy[i]                             # S_i keeps only block i
5             y = y - lam * Si                          # update coordinate i only
6             x = y.copy()                              # Step 3: close the epoch
7             history.append(x.copy())
8         return np.array(history)
9
10    lam = 1.0 / 4.0                                    # = 1/A_ii (exact coord. minimization)
11    hist = cyclic_coordinate_update(np.zeros(3), n_epochs=5, lam=lam)
12    errors = np.linalg.norm(hist - x_star, axis=1)
13    for n, (x_n, e) in enumerate(zip(hist, errors)):
14        print(f"epoch {n}: x={x_n.round(4)}, ||x_n-x*||={e:.4f}")
15    # epoch 0: [0. 0. 0.] , 2.9580
16    # epoch 1: [1.5 1.875 2.1563] , 1.1220
17    # epoch 2: [0.4922 1.5879 2.48 ] , 0.0905    -> x* = [0.5 1.5 2.5]
```

Listing 4: Algorithm 10, Steps 2–3, and results on $Ax = b$

Conclusion: The Book's Own Words

The Book's Own Conclusion (verbatim themes)

“The Krasnosel'skiĭ–Mann iteration is the most popular algorithm to solve the fixed point problems. Its role in the construction and analysis of the iterative algorithms, especially operator splitting methods, in optimization has recently paid increasing attention. Relaxation and inertia are two main factors that affect the convergence speed of the Krasnosel'skiĭ–Mann iteration. Although some progress has been made on the range and selections of the inertial and relaxation parameters, there are few results on the optimal choices. Much of the space remains to be explored for the Krasnosel'skiĭ–Mann iteration.”

Two Open Problems the Book Leaves Us With

- There is still no complete theory for the *optimal* choice of relaxation parameters λ_n (Chapter 7) or inertial parameters (Chapters 5–6) — only ranges and sufficient conditions are known.
- Much of the design space of KM-type methods (how to combine relaxation, inertia, and coordinate splitting) remains unexplored, especially in the large-scale / distributed regime studied in this chapter.

Conclusion: The Arc of the Whole Book

The Arc of the Whole Book, in One Slide

- 1 **Ch. 3 – The KM iteration.** $x_{n+1} = (1 - \lambda_n)x_n + \lambda_n T(x_n)$ for nonexpansive T : the basic machine for finding fixed points, with weak convergence under mild conditions.
- 2 **Ch. 4 – Operator splitting as KM.** Forward–backward, Douglas–Rachford, ADMM, and other splitting methods are all revealed to be instances of the KM iteration for a suitably built nonexpansive operator — a unifying lens.
- 3 **Ch. 5 – Inertial KM.** Adding a momentum/inertial term accelerates convergence, echoing Nesterov-type acceleration in smooth convex optimization.
- 4 **Ch. 6 – Multi-step inertial KM.** Using several past iterates (not just one) for the inertial term, generalizing Chapter 5's single-step inertia.
- 5 **Ch. 7 – Relaxation parameters.** How to choose (and adapt) λ_n to speed up convergence, with sharper rates under extra structure (e.g. strong monotonicity).
- 6 **Ch. 8 – Coordinate-update applications (this chapter).** Cyclic and asynchronous-parallel coordinate updates let the whole KM framework scale to problems where x has millions of coordinates and updating all of them at once is infeasible.

Conclusion: Takeaway

Takeaway

Every method in this book, from the plain KM iteration to ARock's asynchronous coordinate updates, is built from the same two ingredients: a nonexpansive operator T (or its residual $S = \text{Id} - T$) and a carefully chosen step (relaxation, inertia, or which coordinate to touch next). The frontier is tuning these ingredients optimally — and, as the authors note, that frontier is still wide open.